SecFed: A Secure and Efficient Federated Learning Based on Multi-Key Homomorphic Encryption

Yuxuan Cai, Wenxiu Ding, *Member, IEEE*, Yuxuan Xiao, Zheng Yan, *Senior Member, IEEE*, Ximeng Liu, *Senior Member, IEEE*, and Zhiguo Wan, *Member, IEEE*

***Abstract*—Federated Learning (FL) is widely used in various industries because it effectively addresses the predicament of isolated data island. ~~However~~, eavesdroppers is capable of inferring user privacy from the gradients or models transmitted in FL. Homomorphic Encryption (HE) can be applied in FL to protect sensitive data owing to its computability over ciphertexts. ~~However~~, traditional HE as a single-key system cannot prevent dishonest users from intercepting and decrypting the ciphertexts from cooperative users in FL. Guaranteeing privacy and efficiency in this multi-user scenario is still a challenging target. ~~In this article~~, we propose a secure and efficient Federated Learning scheme (SecFed) based on multi-key HE to preserve user privacy and delegate some operations to TEE to improve efficiency while ensuring security. ~~Specifically~~, we design the first TEE-based multi-key HE cryptosystem (EMK-BFV) to support privacy-preserving FL and optimize operation efficiency. ~~Furthermore~~, we provide an offline protection mechanism to ensure the normal operation of system with disconnected participants. ~~Finally~~, we give their security proofs and show their efficiency and superiority through comprehensive simulations and comparisons with existing schemes. SecFed offers a 3x performance improvement over TEE-based scheme and a 2x performance improvement over HE-based solution.**

***Index Terms*—Federated learning, homomorphic encryption, privacy preservation, trusted execution environment.**

~~I. Introduction~~

MACHINE Learning (ML) has shown great success in building models from a tremendous number of training data in various application scenarios, such as computer vision [1], natural language processing [2] and recommender systems [3]. Especially in medical field, ML can extract important information from pathological images to help doctors make a diagnosis. ~~However~~, due to privacy concerns and government regulations, such as GDPR (General Data Protection Regulation). ML often leads to unsatisfactory results for not enough data is available to train well-performing and sufficiently generalized models. ~~Also~~, data owners are reluctant to share their data with others, so it is often difficult for data to flow, which results in the dilemma as "isolated data island". ~~To overcome this issue~~, Federated Learning (FL) first proposed by Google supports multiple entities to jointly train the model by collecting model parameter updates, which are trained by local data without exposing personal raw data. ~~In such multi-user scenarios~~, FL protects users' local data and also leverages the computing resources of mobile devices.

~~However~~, existing research [4] shows that privacy may also be leaked from the gradients or models transmitted in plaintext. Several privacy-preserving techniques such as Secure Multi-party Computation (SMC), Differential Privacy (DP), Trusted Execution Environment (TEE) and HE are applied to FL to protect the gradients or models from attackers and untrusted servers. SMC achieves confidentiality by constructing secure interaction protocols. But SMC increases the computational overhead in central server as well as decreases the efficiency of communication. DP adds noise to gradients or models so that attackers cannot obtain valid information even after intercepting the data. ~~However~~, FL accuracy would be affected as the noise cannot be controlled effectively. TEE provides hardware-level security protection, but constructing this security environment often brings additional overhead. ~~Furthermore~~, due to the requirement of computing on ciphertext, using lightweight cryptographic algorithms in the context of FL tasks would expose raw data to the third-party server, thereby diminishing security and increasing the risk of privacy exposure. Owing to the capability of computing over encrypted data [5], [6], [7], HE can guarantee the privacy of data from being disclosed to attackers without the loss of accuracy. Some privacy-preserving FL frameworks using HE have been proposed sequentially in recent years, but they still faces some challenges in three aspects: privacy, robustness and efficiency.

~~First~~, HE-based FLs leak the privacy to collaborated entities if single-key HE (SHE) is applied, as in scheme [8], where entities can decrypt others' ciphertext. Multiple entities contribute their gradients or models for training. But if all users apply the same

Manuscript received 20 January 2023; revised 21 November 2023; accepted 22 November 2023. Date of publication 29 November 2023; date of current version 11 July 2024. The work was sponsored by the National Natural Science Foundation of China under Grants 61802293 and 62072351, in part by the Natural Science Basic Research Program of Shaanxi under Grant 2023-JC-YB-500, in part by the National Postdoctoral Program for Innovative Talents under Grant BX20180238, in part by the Project funded by China Postdoctoral Science Foundation under Grant 2018M633461, in part by the Open Research Project of Zhejiang Lab under Grant 2021PD0AB01, and in part by the 111 Project under Grant B16037. *(Corresponding author: Wenxiu Ding.)*

Yuxuan Cai, Wenxiu Ding, Yuxuan Xiao, and Zheng Yan are with the School of Cyber Engineering, Xidian University, Xi'an 710071, China (e-mail: 21151213618@stu.xidian.edu.cn; wxding@xidian.edu.cn; xuanXY1105@163.com; zhengyan.pz@gmail.com).

Ximeng Liu is with the College of Computer Science and Data Science, Fuzhou University, Fuzhou 350025, China (e-mail: snbnix@gmail.com).

Zhiguo Wan is with the Zhejiang Laboratory, Hangzhou, Zhejiang 311121, China (e-mail: wanzhiguo@zhejianglab.com).

Digital Object Identifier 10.1109/TDSC.2023.3336977

1545-5971 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Anhui University. Downloaded on March 02,2026 at 02:25:13 UTC from IEEE Xplore. Restrictions apply.

key for encryption, then malicious entities can intercept and decrypt the data from others. In this multi-user scenario, each user is not fully trusted and it is possible for them to steal others' data, so the multi-key HE (MHE) should be applied for FL.

~~Second~~, offline problem needs to be solved. To achieve higher accuracy, more entities and more data are expected to contribute to collaborated training. Multi-key HE can cope with privacy challenges in multi-user scenarios, but it increases the difficulty of data management when some users are offline. The offline of any user will cause the failure of others' decryption for gaining aggregated result in scheme [9]. ~~Hence~~, how to solve the offline problem is still a challenge in HE-based FL.

~~Third~~, HE incurs high computation overhead. Compared with other techniques, HE achieves data privacy without no sacrifice of accuracy and high operation overhead. But it still incurs computation overhead compared to computing directly on plaintext, especially in the case with a great number of users. How to improve the efficiency for practical application is still an open issue.

~~In order to meet the above challenges~~, we first construct multi-key HE cryptosystem named EMK-BFV, which is based on the BFV cryptosystem [10]. We transfer some time-consuming operations in HE to TEE for improving computational efficiency. ~~Then~~, we deploy EMK-BFV into FL and design the corresponding user offline protection mechanism through the cooperation with TEE to improve the robustness of FL system. The contributions of this paper can be summarized as follows:

- We propose an innovative and robust privacy-preserving FL framework named SecFed. It incorporates a novel and efficient multi-key HE, EMK-BFV, along with a flexible offline protection mechanism to enable collaborating participants to upload their data under different keys in order to protect data privacy while ensuring the uninterrupted functionality of the system even when up to $N-2$ participants are disconnected.
- We propose the first TEE-based multi-key HE cryptosystem, called EMK-BFV, to enhance the data security of FL. It reforms BFV by improving its efficiency and security in multi-user scenarios because we migrate some time-consuming confidential operations in original BFV to TEE.
- We analyze the security of SecFed including EMK-BFV and evaluate its efficiency through comparison. We conduct simulation-based experiments based on real data to test the performance of SecFed. Comparation between SecFed and TEE-based FL and HE-based FL highlights the superior performance of SecFed in terms of efficiency and privacy protection.

The rest of this paper is organized as follows. In Section II, we introduce some FL schemes based on privacy-preserving technologies. Our preliminaries and model design are presented in Section III. In Section IV, we elaborate on the construction process of EMK-BFV and SecFed. In Section V, we present the security proofs and conduct experiments to show the performance of EMK-BFV and SecFed. ~~Finally~~, we conclude our work in Section VI.

~~II. Background and Related Work~~

Different from centralized ML, FL enables involved users to exchange gradients or models rather than raw data for collaborative training, which solves the issue of isolated data island. But there still exist some attacks towards the gradients or models [11]. Some research have been conducted to further enhance the security and privacy of user data and models in FL.

A. SMC-Based Federated Learning

SMC enables users to jointly compute a model or a function over their inputs while keeping those inputs private [12]. SMC technique can be applied to implement secure transmission and aggregation of gradients or models. While work [13] revealed the fact that it is difficult for secure aggregation to ensure privacy integrity in the presence of malicious servers. But there are still some works that proposed solutions based on semi-trusted servers. Bonawitz et al. [14] ~~first constructed the SMC protocol to achieve secure aggregation, and the authors constructed a privacy computation process using double masking~~. ~~However,~~ the users involved in the SMC protocol have to communicate and connect with all other users, which is inefficient. Bell et al. [15] improved on work [14] by modifying the way users communicate with each other. Instead of needing to communicate with other $N-1$ users, a particular user communicates with $log(N)$ users. However this reduction in communication overhead is achieved by maintaining new data structures, which introduces additional storage overhead. The work [16] achieved secure aggregation of gradients through secret sharing. ~~However,~~ it comes with additional communication rounds that exchange necessary parameters for secure aggregation. These additional communications severely affect the model convergence speed in case of a poor communication environment. FATE [17] used masking techniques to protect model, in which users need to negotiate between themselves to generate masks. The interrelated masks ensure the security of model aggregation and the correctness of decryption. But FATE introduces the overhead of masking generation and prolongs running time for users with constrained resources. ~~Moreover~~, any loss of mask will cause the decryption to fail. These indicate that SMC-based FL schemes have certain disadvantages in terms of communication overhead and user computation overhead.

B. DP-Based Federated Learning

DP preserves privacy by adding noise to personal sensitive data [18]. There are two methods to apply DP to FL for protecting users' data. One approach is to add noise to user's dataset before local training, such as DPFedAvg-GAN [19] and NbAFL [20]. These schemes used dataset with noise to train model, thus making it impossible for attackers to infer dataset from model. ~~However~~, the noise negatively affects the convergence speed of the model and degrades the model accuracy. Another way is adding noise to parameters in model or directly to gradients on the user side. Abidi et al. [21] added noise to model iteration to protect the privacy dataset. ~~However~~, the FL aggregation process

Authorized licensed use limited to: Anhui University. Downloaded on March 02,2026 at 02:25:13 UTC from IEEE Xplore. Restrictions apply.

causes superimposition of noise, which is difficult to be removed by users and reduces the accuracy of the model. Hu et al. [22] also added noise to the model during the iterative process. Different from [21], the noises added by different users in scheme [22] are not superimposed as they cancel out each other during the aggregation process. However, the model data are also affected by this offsetting effect, which reduces the accuracy. These show that DP-based schemes have a negative on accuracy.

C. TEE-Based Federated Learning

TEE spends extra time to create and maintain a secure environment for programs running by encrypting memory. As in scheme [23] and [24], secure memory ensured data privacy, but introduced additional overhead. In work [23], the authors used TEE to protect the prediction process, which avoided moving the complex training process into secure memory. However, too much prediction data will seriously increase the time overhead. Modal et al. [24] migrated the aggregation process of FL into TEE for execution, which protected user privacy. But the frequent data interactions between memories lead to a dramatic increase in the cost of secure memory, which incurs a larger time overhead. In order to avoid excessive use of secure memory in schemes [23] and [24], Fan et al. [25] used a greedy algorithm to optimize the model to make TEE handle less data and avoided expensive paging mechanism. However, the use of greedy algorithm increases the computation overhead of the regular part of the program and does not achieve significant results. In summary, how to use TEE reasonably to avoid additional time overhead and to achieve a balance between efficiency and security remains an important challenge.

D. HE-Based Federated Learning

HE provided the capability of computation over ciphertexts, which had been widely applied in FL for protecting data privacy. Based on the number of keys used in HE, we could classify them into single-key HE-based FL schemes and multi-key HE-based FL. They achieved privacy protection to some extent, but there are still some privacy issues to be solved.

1) FL Based on Single-Key HE: Some existing work simply applied single-key HE for outsourcing the gradients or models. Paillier cryptosystem as a popular additive homomorphic encryption is widely used for protecting gradients or models in horizontal FL. Yang et al. [26] used Paillier to protect models while utilized FPGA to optimize the encryption, thus improving the operational efficiency of HE-based schemes. BatchCrypt [27] proposed an encoding method that could carry more messages, enabling Paillier to encrypt more gradients information at one time. Madi et al. [28] proposed verifiable privacy-preserving FL using Paillier to protect gradients, in which the homomorphic nature of HE was used to perform ciphertext computation and authentication of server operations. Stripelis et al. [29] then used the CKKS cryptosystem [30] to protect model. In vertical FL, Wei et al. [31] utilized HE to protect the transmission of features between users and server, and realized a secure federated bayesian algorithm based on the vertical distribution of data.

TABLE I
COMPARISONS OF EXISTING WORK

Scheme	PT	MP	OP	NLA	CC
[14]	SMC	✔	✘	✔	H
[15]	SMC	✔	✔	✘	M
[16]	SMC	✔	✘	✔	M
[17]	SMC	✔	✘	✔	M
[19]	DP	✘	✔	✘	L
[20]	DP	✘	✔	✘	L
[21]	DP	✘	✔	✘	L
[22]	DP	✔	✘	✘	L
[24]	TEE	✔	✘	✔	H
[25]	TEE	✔	✘	✔	H
[26] [27] [28] [29] [31] [32]	SHE	✘	-	✔	L
[33]	MHE	✔	✘	✔	M
[34]	MHE	✔	✔	✘	M
[35]	MHE	✔	✘	✔	H
[9]	MHE	✔	✘	✔	H
SecFed	MHE	✔	✔	✔	L

Notes: PT: Privacy Technique; MP: Multi-user Privacy; OP: Offline Protection; NLA: No Loss in Accuracy; CC: Computation Cost; ✔: Support; ✘: Not Support; H: High; M: Medium; L: Low.

The HE algorithm was also introduced to protect the privacy of transfer learning [32]. However, in these schemes, all users hold one pair of keys for data outsourcing and sharing, which is vulnerable to the attacks of malicious users. Single-key HE is not suitable for multi-user scenarios in FL.

2) FL Based on Multi-Key HE: To satisfy the requirements of multi-user scenarios, multi-key HE is applied to protect the security of the gradients or models in FL. In scheme [33], different noises were added to users' private keys during key generation. Hence, users outsourced their data with various private keys. However, if a user is offline, it results in decryption failure as noise cannot be collected correctly. xMK-CKKS [34] realized a multi-key CKKS to protect the outsourced data in FL. Each user encrypted its model with a different key, but CKKS introduces high computation overhead for resource-constrained users. DTAHE [35] introduced a threshold mechanism to achieve privacy protection among multiple users. However, the threshold value cannot be changed in the process of FL, which lacks certain flexibility. Park et al. [9] divided one private key into multiple keys for users. The same public key was used for encryption while the partial private key was for partial decryption. The partial decryption results could be aggregated for obtaining the final result. However, this solution cannot cope with user offline problem.

Through comparisons of existing work shown in Table I, we can determine that SecFed can solve offline problem, and achieve high accuracy and efficiency. But other schemes cannot work well in both security and performance aspects.

Authorized licensed use limited to: Anhui University. Downloaded on March 02,2026 at 02:25:13 UTC from IEEE Xplore. Restrictions apply.

TABLE II
NOTATION DESCRIPTION

Notation	Description
$f(x)=x^d+1$	The monic polynomial
q	The ciphertext space coefficient modulo
t	The plaintext space coefficient modulo
n	The number of users
R_q	The ciphertext space
R_t	The plaintext space
ξ	The discrete Gaussian distribution on a quotient ring
$secret$	The master key
sk_i	The private key of user i
pk_i	The public key of user i
rk_i	The recovery key of user i
e	The noise sampled from a discrete Gaussian distribution
a	The random number sampled from the ciphertext space
μ	The random number sampled from a discrete Gaussian distribution
$result_i$	The calculation result of user i
pt_i	The plaintext for user i
pt_n^{agg}	The aggregation of pt_1 to pt_i
ct_i	The ciphertext for user i
$ct_i^{'}$	The supplementary ciphertext for user i
ct_k^{agg}	The partially aggregated ciphertext
$ct_k^{agg'}$	The partially aggregated ciphertext after noise refresh
ct_n^{agg}	The aggregated ciphertext
ID_i	The ID of user i

III. PRELIMINARIES AND SYSTEM MODEL

A. Notations and Preliminaries

1) Notations: For easy presentation, Table II first lists the parameters and related symbols required in this paper.

2) Homomorphic Encryption - BFV: We will describe the construction and operation process of the BFV cryptosystem to facilitate a comparison with the operation process of EMK-BFV.

Consistent with EMK-BFV, we use $R_q=\mathbb{Z}_q[x]/f(x)$ to denote the plaintext space and $R_t=\mathbb{Z}_t[x]/f(x)$ to denote the ciphertext space, which $q>>t$. The degree d of the irreducible polynomial is typically set to a power of two, ranging from 1024 to 16384, to balance computational efficiency with cryptographic security. The discrete Gaussian distribution $D_{\mathbb{Z},\sigma}$ over integers assigns to each $x\in\mathbb{Z}$ a probability proportional to $exp(-\pi|x|^2/\sigma^2)$. Gaussian noise with a carefully selected standard deviation, typically in the range of small to moderate values (3.2–3.5), is used to balance encryption security and computational accuracy. Use the discrete Gaussian distribution $D_{Z,\sigma}$ to define the distribution ξ of R_q. And $e\leftarrow\xi$ to denote that e is sampled uniformly from ξ. Let $\Delta=\lfloor q/t\rfloor$ and $q=\Delta t+r_t(q)$ where $r_t(q)=q \bmod t$. Let λ be the security parameter. We mainly use seven functions including $BFV.skGen$, $BFV.pkGen$, $BFV.rlkGen$, $BFV.enc()$, $BFV.dec()$, $BFV.add()$, $BFV.noiseRef()$ and $BFV.mul()$ to denote the main process of BFV cryptosystem.

1) Key Generation:

- $BFV.skGen(1^\lambda)$: Randomly sample $s\leftarrow R_2$ and set $sk=s$.
- $BFV.pkGen(sk)$: Set $s=sk$, sample $a\leftarrow R_a, e\leftarrow\chi$ and output:

$$pk=(pk[0],pk[1])=([-(as+e)]_q,a). \tag{1}$$

- $BFV.rlkGen(sk,w)$: Slice the ciphertext generated by multiplication into parts of small norm by choosing a base T, and set $l=log_T^{(q)}$. Then output:

$$rlk=[([-(a_i\cdot s+e_i)+T^i]_q,a_i):i\in[0\cdots l]]. \tag{2}$$

2) Encryption and Decryption:

- $BFV.enc(pk,m)$: For $m\in R_t$, sample $\mu,e_1,e_2\leftarrow\chi$ and output:

$$\begin{aligned}ct&=(ct[0],ct[1])\\&=([pk[0]\cdot\mu+e_1+\Delta\cdot m]_q,[pk[0]\cdot\mu+e_2]_q).\end{aligned} \tag{3}$$

- $BFV.dec(sk,ct)$: Set $s=sk$, Decrypt ciphertext by computing:

$$\left[\left\lfloor\frac{t\cdot[ct[0]+ct[1]\cdot s]_q}{q}\right\rceil\right]_t. \tag{4}$$

3) Aggregation:

- $BFV.add(ct_i,ct_j)$: Aggregate two ciphertext to get ct^{agg}:

$$\begin{aligned}ct^{agg}&=(ct^{agg}[0],ct^{agg}[1])\\&=(ct_i[0]+ct_j[0],ct_i[1]+ct_j[1]).\end{aligned} \tag{5}$$

4) Noise Refresh:

- $BFV.noiseRef(sk,pk)$: Modulus switching and Bootstraping [36] can be applied to BFV cryptosystems to control the increase of noise.

5) Multiplication:

- $BFV.mul(ct_i,ct_j)$: Implement the multiplication of ct_i and ct_j by computing:

$$ct[0]=\left[\left\lfloor\frac{t\cdot ct_i[0]ct_j[0]}{q}\right\rceil\right]_q, \tag{6}$$

$$ct[1]=\left[\left\lfloor\frac{t}{q}(ct_i[0]ct_j[1]+ct_i[1]ct_j[0])\right\rceil\right]_q, \tag{7}$$

$$ct[2]=\left[\left\lfloor\frac{t\cdot ct_i[1]ct_j[1]}{q}\right\rceil\right]_q. \tag{8}$$

B. System Model and Security Model

1) System Model: Our proposed system mainly comprises two types of entities. Their functions and characteristics are as follows:

1) *User:* The users are the service requesters and consumers, who outsource their personal data and then gain the learning results from CS.
2) *Cloud Server (CS):* CS performs FL process to serve users. CS consists of two parts: General Part (GP) and Trusted

Authorized licensed use limited to: Anhui University. Downloaded on March 02,2026 at 02:25:13 UTC from IEEE Xplore. Restrictions apply.

TABLE III
THREAT MODEL ANALYSIS

Threat Types	Initiator	Attack Feature	Loss
Eavesdropping data	Eavesdroppers/Users	Eavesdroppers and users can monitor the channel and intercept the transmitted data	Users' datasets and model information
Collusion attack	Users and GP	GP colludes with users to decrypt the data of others	User's datasets
Malicious leakage	GP	GP intentionally leaked user data for profit	Users' dataset

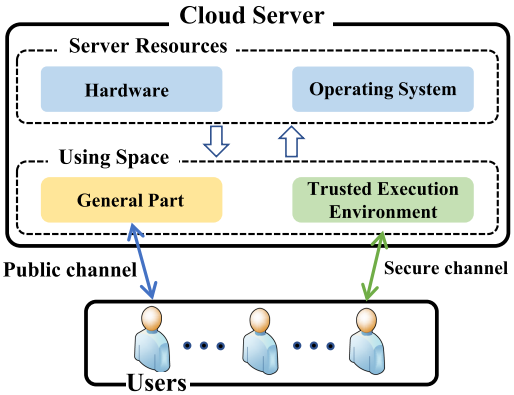


Fig. 1. System model.

Execution Environment (TEE). GP is an untrusted part of CS, just like a regular server, while TEE is the secure memory in CS.

In SecFed, users mainly interact with the GP in CS through the public channel to share data. Key transmissions are carried out between users and TEE, which are secured through a key exchange protocol. The system entities and their relationships are shown in Fig. 1.

2) Security Model: ~~In our system~~, TEE is regarded as fully trusted and follows protocol honestly [37], which would never collude with other entities. GP and users are curious-but-honest. ~~That is~~, they are curious about others' private data, but strictly follow the protocol design. In our security model settings, honest but curious users may conspire with GP to steal other users' private data, but they do not launch poisoning attacks. And some existing methods [38], [39], such as training adversarial sample models can be used to counteract the effects of these attacks. ~~In brief~~, defending against poisoning attacks is not the focus of our research. Our primary objective is to ensure the uninterrupted operation of SecFed (system robustness), particularly to avoid disruptions due to user offline scenarios.

~~Generally~~, there are three main threats to users' data. ~~First~~, during the uploading and downloading process, a malicious eavesdropper/user may intercept the transmitted data. ~~Second~~, malicious users collude with GP to probe into sensitive data. ~~Third~~, data leakage can occur on GP. The GP can gain extra profit by leaking the user's private data. ~~Therefore~~, we need to resist the three kinds of attacks. A summary of the security model is shown in Table III.

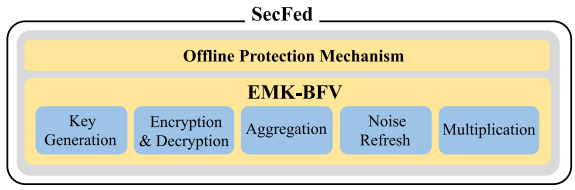


Fig. 2. ~~Architecture of SecFed.~~

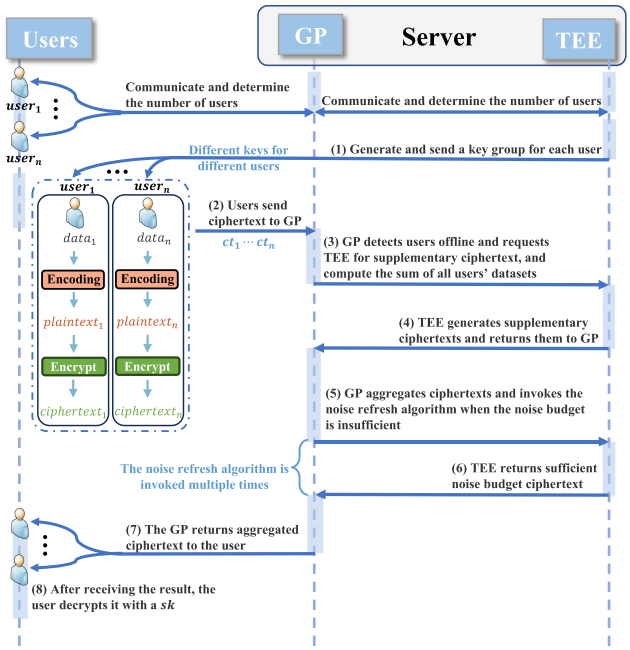


Fig. 3. ~~Flowchart of SecFed~~.

IV. SECFED FRAMEWORK

~~In this section~~, we present the design of SecFed framework. Section IV-A gives the detailed structure of EMK-BFV, followed by the details of SecFed in Section IV-B.

Fig. 2 illustrates the architecture of SecFed, which mainly consists of two components operating on TEE hardware: EMK-BFV and Offline Protection Mechanism. Multi-key HE often encounters issues with decryption failure due to user offline. ~~Therefore~~, the introduction of TEE not only enhances the efficiency of EMK-BFV, but also supports the operation of the Offline Protection Mechanism, thereby improving the robustness of SecFed.

Authorized licensed use limited to: Anhui University. Downloaded on March 02,2026 at 02:25:13 UTC from IEEE Xplore. Restrictions apply.

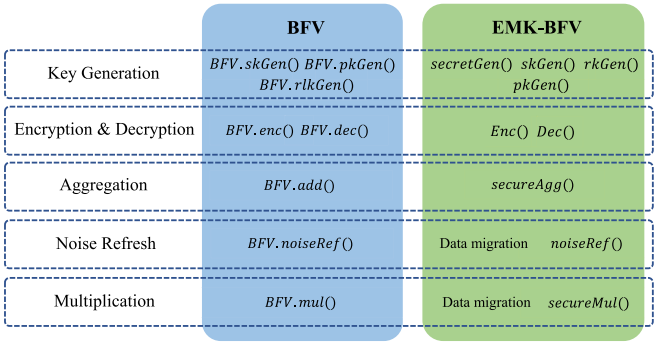


Fig. 4. Overall and details of BFV and EMK-BFV.

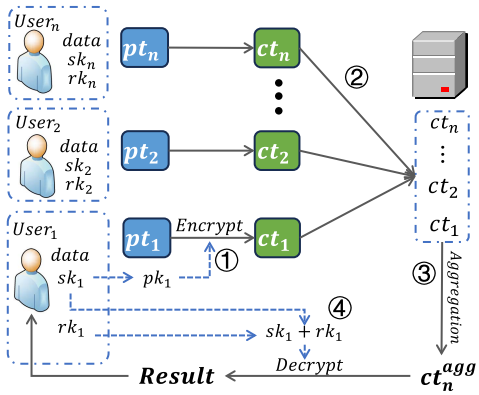


Fig. 5. Encryption and decryption process.

The process of SecFed is shown in Fig. 3. Before starting FL, users and server (GP and TEE) first exchange some parameters to determine basic information, such as the number of users. ~~In step (1),~~ TEE generates a key set for each user and transmits them to users through a secure channel. ~~In step (2),~~ users encrypt personal information using their own public key and send ciphertext to GP. ~~At the same time~~, information about the size of dataset is sent synchronously, which will be used for the calculation of the weights during the aggregation. ~~In step (3),~~ GP detects disconnected users and requests TEE for supplementary ciphertext to compute the sum of all users' datasets. Upon receiving the request from the GP, TEE generates the missing supplementary ciphertexts and returns them to the GP as step (4). From step (5) to (7), GP begins to aggregate after receiving ciphertexts from all users, and calls TEE to refresh the noise when the noise budget is insufficient. This invocation is made several times during the aggregation process. ~~Finally,~~ the GP obtains the aggregated ciphertext ct_n^{agg} and returns it to users. ~~In step (8),~~ each user receives the same ct_n^{agg} and decrypt it by sk_i and rk_i.

A. Design of EMK-BFV

To satisfy the requirements of multi-user scenario, we first reconstruct an efficient multi-key homomorphic encryption based on BFV cryptosystem: EMK-BFV. In Fig. 4, we present the differences of traditional BFV and our EMK-BFV. The detailed structure of EMK-BFV are as follows:

1) Key Generation: We first design multi-key generation algorithm to meet the challenges in multi-user scenarios while achieving key confidentiality. We construct four functions: $secretGen()$, $skGen()$, $rkGen()$, $pkGen()$ and perform them in TEE to generate keys for users. Algorithm 1 shows the execution of each function.

- $\boldsymbol{secretGen(R_2) \rightarrow secret}$

Randomly select $secret$ from a polynomial quotient ring with coefficients 0 or 1 as master key: $secret \leftarrow R_2 = \mathbb{Z}_2[x]/f(x)$. We use $secret$ as the master key of the system, while $f(x)$ is chosen to ensure the security of $secret$.

- $\boldsymbol{skGen(secret) \rightarrow [sk_1, sk_2, \ldots, sk_n]}$

We use the generated master key $secret$ to generate private keys sk_i for n users respectively. The master key is randomly divided using polynomial addition and subtraction rules such that:

$$secret = s_1 + s_2 + \cdots + s_n. \tag{9}$$

The coefficients of $s_1, s_2, \ldots, s_n$ are also 0 or 1. We set $sk_i = s_i$, and share the private key sk_i to each user, respectively.

- $\boldsymbol{rkGen(secret, [sk_1, \ldots, sk_n]) \rightarrow [rk_1, \cdots rk_n]}$

~~After aggregation~~, the aggregated ciphertext can be regarded as a ciphertext encrypted by $secret$. ~~Therefore~~, we generate recovery key rk for each user to ensure that the aggregated ciphertext can be decrypted correctly. Generate recovery keys rk_i for individual users using their private sk_i and master keys $secret$:

$$rk_1 = secret - sk_1, \ldots, rk_n = secret - sk_n. \tag{10}$$

With rk, the user can recover the $secret$ while no other user's sk can be inferred. Finally we send $[rk_1, rk_2, \ldots, rk_n]$ to users for decrypting the ciphertext after aggregation.

- $\boldsymbol{pkGen([sk_1, \ldots, sk_n], \xi, q) \rightarrow [pk_1, \ldots, pk_n]}$

We design pk based on each user's sk to ensure that each user can encrypt the data in the correct way. First randomly select element $a \leftarrow R_q = \mathbb{Z}_q[x]/f(x)$ and noise $e_i \leftarrow \xi$, then generate the public key pk_i for user i:

$$pk_i = (pk_i[0], pk_i[1]) = ([-(a \cdot sk_i + e_i)]_q, a). \tag{11}$$

Then Users can generate interference terms that are mathematically equivalent to the plaintext by superimposing pk with the plaintext. And the user can use sk to eliminate the interference terms when decrypting.

~~Through the above functions~~, the system master key $secret$ and each user's keys can be obtained. Then we generate encryption parameters $\mu \leftarrow R_2 = \mathbb{Z}_2[x]/f(x)$ for users to ensure the consistency of encryption. The user i finally receives $[pk_i, sk_i, rk_i, \mu]$.

2) Encryption & Decryption: We design $Enc()$ and $Dec()$ to realize the homomorphic property, which are presented in Algorithm 2. The two functions are executed by the user as shown in Fig. 5.

- $\boldsymbol{Enc(pk_i, pt_i, \mu, q, t, \xi) \rightarrow ct_i}$

The user i first randomly samples $e_{i1}, e_{i2} \leftarrow \xi$. Then we use interference terms $(pk[0] \cdot \mu)$ generated by pk and e_{i1} to

Authorized licensed use limited to: Anhui University. Downloaded on March 02,2026 at 02:25:13 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: ~~Key Generation.~~

Function 1.1: $\boldsymbol{secretGen()}$
Input: R_2;
Output: $secret$;
  define a polynomial quotient $R_2 \leftarrow \mathbb{Z}[x]/f(x)$;
  set $secret \leftarrow R_2$;
  return $secret$;
Function 1.2: $\boldsymbol{skGen()}$
Input: $secret$;
Output: $[sk_1, sk_2, \ldots, sk_n]$;
  define a polynomial quotient $R_2 \leftarrow \mathbb{Z}[x]/f(x)$;
  for i=1 to $n-1$:
    set $s_i \leftarrow R_2, secret \leftarrow secret - s_i$;
  end for;
  set $s_n \leftarrow secret, sk_1 \leftarrow s_1, \ldots, sk_{n-1} \leftarrow s_{n-1}$;
  return $[sk_1.sk_2.\cdots, sk_n]$;
Function 1.3: $\boldsymbol{rkGen()}$
Input: $serect, [sk_1, sk_2, \ldots, sk_n]$;
Output: $[rk_1, rk_2, \ldots, rk_n]$;
  for i=1 to n:
    set $rk_i \leftarrow (secret - sk_i)$;
  end for;
  return $[rk_1, rk_2, \ldots, rk_n]$;
Function 1.4: $\boldsymbol{pkGen()}$
Input: $[sk_1, sk_2, \ldots, sk_n], \xi, q$;
Output: $[pk_1, pk_2, \ldots, pk_n]$;
  define $R_q \leftarrow \mathbb{Z}_q[x]$ and Gaussian distribution ξ;
  set $a \leftarrow R_q$;
  for i=1 to n:
    set $e_i \leftarrow \xi, pk_i \leftarrow ([-(a \cdot sk_i + e_i)]_q, a)$;
  end for;
  return $[pk_1, pk_2, \ldots, pk_n]$;

obfuscate the pt to form the first part of the ciphertext $ct[0]$. Also we keep a data structure $(a \cdot \mu + e_{i2})$ in the second part of the ciphertext $ct[1]$, based on which the user can use sk to eliminate the interfering terms to obtain the pt. Due to the RLWE problem, e_{i2} can ensure the security of this data structure. Finally, we use pk to encrypt to obtain the ciphertext $ct_i = (ct_i[0], ct_i[1])$ as shown in step ①, where:

$$\begin{aligned} ct_i[0] &= \left[pk_i[0] \cdot \mu + e_{i1} + \frac{q}{t} pt_i\right]_q \\ &= \left[-a \cdot \mu \cdot sk_i + e_i \cdot \mu + e_{i1} + \frac{q}{t} \cdot pt_i\right]_q, \\ ct_i[1] &= [pk_i[1] \cdot \mu + e_{i2}]_q \\ &= [a \cdot \mu + e_{i2}]_q. \end{aligned} \tag{12}$$

- $\boldsymbol{Dec(sk_i, rk_i, ct_n^{agg}) \rightarrow pt_i^{agg}}$

The GP in server receives the ciphertexts of the users through step ② and aggregates them to obtain the aggregated ciphertext ct_n^{agg}. The aggregated ciphertext is sent to each user for individual decryption. Each user can decrypt the aggregated ciphertext ct_n^{agg} by utilizing sk_i and rk_i as $sk_i + rk_i$. The process of decryption and its correctness is as follows:

After aggregating the ciphertexts from various users with different keys, the interference terms $-a \cdot \mu \sum_{i=1}^{n} sk_i$ in ct_n^{agg} are superimposed:

$$\begin{aligned} ct_n^{agg} &= (ct_n^{agg}[0], ct_n^{agg}[1]) \\ &= \left(\left[-a \cdot \mu \sum_{i=1}^{n} sk_i + e' \right. \right. \\ &\quad \left. \left. + \frac{q}{t} \sum_{i=1}^{n} pt_i \right]_q, [pk_i[1] \cdot \mu + e_{i2}]_q \right). \end{aligned} \tag{13}$$

For the superimposed interference terms, the user i can use its sk_i and rk_i to eliminate them:

$$\begin{aligned} & ct_n^{agg}[0] + ct_n^{agg}[1] \cdot (sk_i + rk_i) \\ &= -a \cdot \mu \sum_{i=1}^{n} sk_i + e' + \frac{q}{t} \sum_{i=1}^{n} pt_i \\ &\quad + a \cdot \mu \cdot (sk_i + rk_i) + e_{i2} \cdot (sk_i + rk_i) \\ &= -a \cdot \mu \cdot secret + e' + \frac{q}{t} \sum_{i=1}^{n} pt_i \\ &\quad + a \cdot \mu \cdot secret + e_{i2} \cdot secret \\ &= e' + \frac{q}{t} \sum_{i=1}^{n} pt_i + e_{i2} \cdot secret. \end{aligned} \tag{14}$$

The user ends up with: $e' + \frac{q}{t} \sum_{i=1}^{n} pt_i + e_{i2} \cdot secret$, in which only the superimposed noise and aggregated plaintext remain. Since $noiseRef()$ is continuously invoked during the aggregation process, e' and $e_{i2} \cdot secret$ will not exceed the security threshold thus not leading to decryption failure. Then each user multiplies it by $\frac{t}{q}$ to get:

$$\frac{t}{q} e' + \frac{t}{q} \frac{q}{t} \sum_{i=1}^{n} pt_i + \frac{t}{q} e_{i2} \cdot secret. \tag{15}$$

Since $q >> t$, the $\frac{t}{q} e'$ and $\frac{t}{q} e_{i2} \cdot secret$ are all decimals, which can be eliminated after rounding. Finally we get $pt_n^{agg} = \sum_{i=1}^{n} pt_i$.

3) Aggregation: We design function $secureAgg()$ to implement aggregation of ciphertexts under mutiple public keys. The process of ciphertext aggregation is mainly done by GP. During the aggregation process, if the noise accumulation exceeds the boundary, the noise refresh function needs to be mobilized. The execution of $secureAgg()$ is shown in Algorithm 3.

- $\boldsymbol{secureAgg([ct_1, ID_1], \ldots, [ct_n, ID_n]) \rightarrow ct_n^{agg}}$

We design aggregation rules to ensure that interfering items and plaintexts in ciphertexts from different users can be overlaid correctly. The correct aggregation result is a precondition for the user to be able to decrypt. After GP receives the users' ciphertexts, it aggregates the first part $ct_i[0]$ of each ciphertext:

$$\sum_{i=1}^{n} ct_i^{agg}[0] = \left[\sum_{i=1}^{n} \left(-a \cdot \mu \cdot sk_i + e_i \cdot \mu + e_{i1} + \frac{q}{t} pt_i \right) \right]_q. \tag{16}$$

Authorized licensed use limited to: Anhui University. Downloaded on March 02,2026 at 02:25:13 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: ~~Encryption & Decryption.~~

Function 2.1: $\boldsymbol{Enc()}$
Input: $pk_i, pt_i, \mu, q, t, \xi$;
Output: ct_i;
  define discrete Gaussian distribution ξ;
  set $e_{i1}, e_{i2} \leftarrow \xi$,
    $ct_i \leftarrow$
    $([pk_i[0] \cdot \mu + e_{i1} + \frac{q}{t} pt_i]_q, [pk_i[1] \cdot \mu + e_{i2}]_q)$;
  return ct_i;
Function 2.2: $\boldsymbol{Dec()}$
Input: sk_i, rk_i, ct_n^{agg};
Output: pt_i^{agg};
  set $pt_i^{agg} \leftarrow$
    $round(ct_n^{agg}[0] + ct_n^{agg}[1] \cdot (sk_i + rk_i) \cdot \frac{t}{q})$;
  return pt_n^{agg};

Algorithm 3: ~~Aggregation~~.

Function3.1: $\boldsymbol{secureAgg()}$
Input: $[ct_1, ID_1], \ldots, [ct_n.ID_n]$;
Output: ct_n^{agg};
  set $ct_n^{agg} \leftarrow (0, ct_1[1])$;
  for i=1 to n:
    if noise budget $<$ safe value:
      set $temp \leftarrow noiseRef(ct_n^{agg}, ID_i)$;
    else:
      set $temp \leftarrow ct_i[0]$;
    end if;
    set $ct_n^{agg}[0] \leftarrow ct_n^{agg}[0] + ct_i[0]$;
  end for;
  set $ct_n^{agg} \leftarrow (0, ct_1[1])$;
  return ct_n^{agg};

If the noise budget is insufficient during aggregation, GP needs to call the noise refresh function $noiseRef()$ executed by TEE.

We finally obtain $ct_n^{agg}[0]$, and then select one second part $ct_i[1]$ of any user's ciphertext ct_i as the second part $ct_n^{agg}[1]$ of the aggregated ciphertext ct_n^{agg}. Because the data structure present in $ct_n^{agg}[1]$ ensures that the user eliminates interfering terms, we need to ensure its invariance.

4) Noise Refresh: In the process of aggregation, the noise will be constantly superimposed. If the superimposed noise exceeds a certain threshold, then the decryption will be failed. Some solutions such as modulus switching techniques are not efficient enough in dealing with noise. ~~Therefore~~, we design a noise refresh function $noiseRef()$ in Algorithm 4 to refresh the noise fast and thoroughly.

- $\boldsymbol{noiseRef(ct_k^{agg}, ID_k) \rightarrow ct_k^{agg'}}$

When GP aggregates the kth ciphertext, if noise budget is insufficient, $noiseRef()$ will be performed in TEE. CS passes the partial aggregated ciphertext ct_k^{agg} and the current identifier of kth ciphertext to TEE. Then TEE constructs two temporary key $pk_{1\to k}$ and $sk_{1\to k}$ for encryption and decryption according

Algorithm 4: ~~Noise Refresh~~.

Function4.1: $\boldsymbol{noiseRef()}$
Input: ck_k^{agg}, ID_k;
Output: $ct_k^{agg'}$;
  define discrete Gaussian distribution ξ;
  set $e_k, e_{k1}, e_{k2} \leftarrow \xi, sk_{1\to k} \leftarrow 0$;
  for i=1 to k:
    $sk_{1\to k} \leftarrow sk_{1\to k} + sk_i$;
  end for;
  set $pt_{1\to k} \leftarrow Dec(sk_{1\to k}, 0, ck_k^{agg})$,
    $pk_{1\to k} \leftarrow ([-(a \cdot sk_{1\to k} + e_k)]_q, a)$,
    $ct_k^{agg'} \leftarrow Enc(pk_{1\to k}, pt_{1\to k}, \mu, q, t)$;
  return $ct_k^{agg'}$;

to the identifier as follows:

$$\begin{aligned} sk_{1\to k} &= sk_1 + sk_2 + \cdots + sk_k, \\ pk_{1\to k} &= (pk_{1\to k}[0], pk_{1\to k}[1]) \\ &= ([-a \cdot sk_{1\to k} + e_k]_q, a). \end{aligned} \tag{17}$$

The discrete distribution ξ and parameter a are stored in secure memory and can be accessed directly by TEE.

After that TEE decrypts ct_k^{agg} using $sk_{1\to k}$ and gets plaintext. The noise has disappeared during the decryption process.

TEE needs to transmit secure ciphertext $ct_k^{agg'}$ with a manageable noise to GP. TEE uses $pk_{1\to k}$ and μ to encrypt the plaintext and then transmits it to GP. μ is also stored in TEE.

5) Multiplication: To implement ciphertext multiplication, we designed function $secureMul()$ in Algorithm 5, which is executed in TEE. This fucntion achieves fast multiplication on the basis of additive homomorphism.

- $\boldsymbol{secureMul(ct_i, ct_j, ID_i, ID_j) \rightarrow [ct_i^{i\cdot j}, ct_j^{i\cdot j}]}$

The multiplication calculation is done entirely by TEE. We perform the multiplication operation directly in secure memory, in plaintext. User i and user j send their ciphertext ct_i and ct_j with identity ID_i and ID_j to TEE. The ciphertext is loaded into secure memory (i.e., TEE) and user's private key is also stored in secure memory. TEE uses the private keys sk_i and sk_j to decrypt the ciphertext and gets pt_i and pt_j. Therefore, TEE can perform multiplication in the plaintext form and get $pt^{i\cdot j}$. Then TEE encrypts it using the public keys of user i and user j, the process is as:

$$\begin{aligned} pt^{i\cdot j} &= Enc_{pk_i}(Dec_{sk_i}(ct_i) \cdot Dec_{sk_j}(ct_j)), \\ pt^{i\cdot j} &= Enc_{pk_j}(Dec_{sk_i}(ct_i) \cdot Dec_{sk_j}(ct_j)). \end{aligned} \tag{18}$$

The user i only needs to utilize sk_i to decrypt without rk_i. This function is called only when ciphertext multiplication is required.

B. Design of SecFed

~~Herein~~, we apply our proposed EMK-BFV in Section IV-A to achieve an efficient and secure FL framework, SecFed. We also design the offline protection mechanism to solve the problem

Authorized licensed use limited to: Anhui University. Downloaded on March 02,2026 at 02:25:13 UTC from IEEE Xplore. Restrictions apply.

Algorithm 5: ~~Multiplication.~~

Function5.1: $secureMul()$
Input: ct_i, ct_i, ID_i, ID_j;
Output: $ct_i^{i\cdot j} ct_j^{i\cdot j}$;
  set $pt_i \leftarrow Dec(sk_i, 0, ct_i), pt_j \leftarrow Dec(sk_j, 0, ct_j)$,
    $pt^{i\cdot j} \leftarrow pt_i \cdot pt_j$,
    $ct_i^{i\cdot j} \leftarrow Enc(pk_i, pt^{i\cdot j}, \mu, q, t, \xi)$,
    $ct_j^{i\cdot j} \leftarrow Enc(pk_j, pt^{i\cdot j}, \mu, q, t, \xi)$;
  return $ct_i^{i\cdot j}, ct_j^{i\cdot j}$;

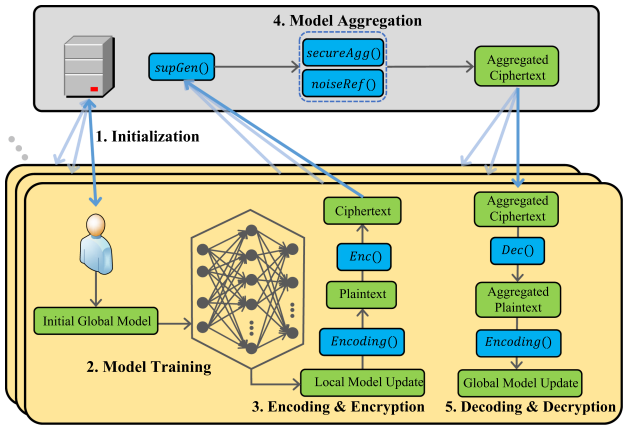


Fig. 6. SecFed process.

of users dropout in SecFed. And we propose the implementation of system initialization and the method of user encoding and decoding. The running process of SecFed is shown in Fig. 6.

1) Initialization: During the initialization phase, two operations are performed. The first is to determine the importance of each user's dataset, which we denote by δ. ~~Secondly~~ TEE generates keys by key generation in EMK-BFV and initial global model.

We design function $impGen()$ in Algorithm 6 to be executed by both CS and users to compute δ. CS sends keys over a secure channel or key exchange protocol and initial global model over a common channel to users.

- $\boldsymbol{impGen(size_1, \ldots, size_n) \rightarrow [\delta_1, \ldots, \delta_n]}$

Before proceeding with the FL procedure, each user needs to determine the weight of their own dataset in the datasets of all users. Each user sends their size of dataset $size_i$ to the GP, and GP finally receives $[size_1, \ldots, size_n]$, and then calculates δ_i for each user:

$$\delta_i = \frac{size_i}{\sum_{i=1}^{n} size_i}, \tag{19}$$

where δ_i indicates the weights of each user's dataset for satisfying GP's need.

2) ~~Model Training.~~ Users train the model with private dataset based on the initial global model. Each user gets the local model update after training.

Algorithm 6: ~~System Initialization.~~

Function 6.1: $impGen()$
Input: $size_1, size_2, \ldots, size_n$;
Output: $\delta_1, \delta_2, \ldots, \delta_n$;
  set $count \leftarrow 0$;
  for i=1 to n:
    $count \leftarrow count + size_i$;
  end for;
  for i=1 to n:
    $\delta_i \leftarrow \frac{size_i}{count}$;
  end for;
  return $\delta_1, \delta_2, \ldots, \delta_n$;

3) ~~Encoding & Encryption:~~ After getting the local model update, the user needs to encode and encrypt it. The purpose of encoding is to map the data into plaintext space R_q that can be encrypted. ~~For this reason~~, we design function $Encoding()$ to accomplish this process, which is as shown in Algorithm 7. ~~After that,~~ the user encrypts the plaintext to obtain the ciphertext by $Enc()$ in EMK-BFV. The ciphertext and user's ID are sent to the GP.

- $\boldsymbol{Encoding(W, message_i, \delta_i, \eta) \rightarrow pt_i}$

The local model update is expressed as a series of parameters: $W = [w_1, w_2, \ldots, w_m]$, where w_i is a floating number. Since the encrypted object of SecFed are integers, we need to map the W to the plaintext space and at the same time ensure that the computed result can be mapped back correctly. ~~Apply the impact factor δ_i to local model update and get~~:

$$\delta_i \cdot W = [\delta_i \cdot w_1, \delta_i \cdot w_2, \ldots, \delta_i \cdot w_m]. \tag{20}$$

~~In this way~~, we can correctly quantify the importance of the model. The $message = \delta_i \cdot W$ is an m-dimensional floating-point vector. We convert floating-point to integers by η, after which these data can be used as the coefficient of the univariate equation of $(m-1)$ degree:

$$\eta \cdot \delta_i \cdot w_1 \cdot x^0 + \eta \cdot \delta_i \cdot w_2 \cdot x^1 + \cdots + \eta \cdot \delta_i \cdot w_m \cdot x^{m-1}. \tag{21}$$

If $m-1$ is greater than the polynomial degree, then we use multiple plaintext polynomial storage models. The process of encoding a message into plaintext that can be encrypted is as follows:

$$\begin{aligned} &[w_1, w_2, \ldots, w_m,] \rightarrow [\delta_i \cdot w_1, \delta_i \cdot w_2, \ldots, \delta_i \cdot w_m] \\ &\rightarrow [\eta \cdot \delta_i \cdot w_1, \eta \cdot \delta_i \cdot w_2, \ldots, \eta \cdot \delta_i \cdot w_m] \\ &\rightarrow \eta \cdot \delta_i \cdot w_1 \cdot x^0 + \cdots + \eta \cdot \delta_i \cdot w_m \cdot x^{m-1}. \end{aligned} \tag{22}$$

4) Model Aggregation: After GP receives ciphertexts from users, it needs to aggregate ciphertext for model update. But if any offline user is detected, the aggregated result will be incorrect and result in failed decryption in user side. ~~Hence~~, we design an offline protection mechanism $supGen()$ to recover the missing ciphertext.

GP passes offline users' IDs to TEE, after which TEE executes $supGen()$ to generate supplementary ciphertexts and return them to GP. After getting the ciphertexts of all users, the GP

Authorized licensed use limited to: Anhui University. Downloaded on March 02,2026 at 02:25:13 UTC from IEEE Xplore. Restrictions apply.

Algorithm 7: ~~User Operation~~.

Function 7.1: $\boldsymbol{Encoding()}$
Input: $W, message_i, \delta_i, \eta$;
Output: pt_i;
  set $item \leftarrow 0$;
  for i=1 to m:
    $item \leftarrow \delta_i \cdot \eta \cdot w_i, W[i] \leftarrow item$;
  end for;
  for i=1 to m:
    $pt_i \leftarrow pt_i + W[i] \cdot x^{i-1}$;
  end for;
  return pt_i;
Function 7.2: $\boldsymbol{Decoding()}$
Input: pt_n^{agg}, η;
Output: W_n^{agg};
  set $i = 0$;
  for $item$ in pt_n^{agg}:
    $W_n^{agg}[i] \leftarrow \frac{item}{x^i}, i++$;
  end for;
  return $W_n^{agg}[i]$;

performs ciphertext aggregation by executing $secureAgg()$ in EMK-BFV.

During the aggregation process, if the noise budget is insufficient, GP calls $noiseRef()$ in EMK-BFV to refresh noise and gets ciphertext with security noise. The design of function $supEnc()$ in Algorithm 8 is shown as follows:

- $\boldsymbol{supEnc([ID_j \cdots ID_k], \mu, \xi, a, q) \rightarrow [ct'_j \cdots ct'_k]}$

With the IDs of offline users (such as, $[ID_j \cdots ID_k]$), this function can generate supplementary ciphertexts to ensure correct decryption. Because the user's offline will result in the absence of the ciphertext and thus the overlay of interfering terms is not complete. ~~However~~, the user decrypts the aggregated ciphertext by eliminating the complete interfering terms, which will cause the decryption failure. Therefore we need to ensure the completeness of the aggregated ciphertext. For each offline user in $[ID_j, \ldots, ID_k]$, we randomly sample noise from a discrete Gaussian distribution: $e_{j1}, \ldots, e_{k1} \leftarrow \xi$, then use its public key $[pk_j, \ldots, pk_k]$ to generate supplementary ciphertext $(i = j \rightarrow k)$:

$$\begin{aligned} ct'_i[0] &= [pk_i[0] \cdot \mu + e_{i1}]_q, \\ &= \left[-a \cdot \mu \cdot sk_i + e_i \cdot \mu + e_{i1} + \frac{q}{t} \cdot pt_i\right]_q, \\ ct'_i[1] &= [a \cdot \mu + e_{i2}]_q. \end{aligned} \tag{23}$$

The ciphertexts $[ct'_i, \ldots, ct'_j]$ generated for offline users in $[ID_j \cdots ID_k]$ are passed to GP for further calculation.

5) Decoding & Decryption: Each user needs to decrypt and decode the aggregated ciphertext after receiving it. The user uses $Dec()$ in EMK-BFV to decrypt, and then we design function $Decoding()$ in Algorithm 7 to help each user to decode plaintext. After decoding, the user gets the global model update.

- $\boldsymbol{Decoding(pt_n^{agg}, \eta) \rightarrow W_n^{agg}}$

Algorithm 8: ~~Offline Protection Mechanism.~~

Function 8.1: $\boldsymbol{supEnc()}$
Input: $[ID_j, \ldots, ID_k], \mu, \xi, a$;
Output: $ct'_j, \ldots, ct'_k$;
  for i=j to k:
    set $e_{i1}, e_{i2} \leftarrow \xi$;
  end for;
  set $[pk_j, \ldots, pk_k]$ according to $[ID_j, \ldots, ID_k]$;
  for i=j to k:
    set $ct'_i \leftarrow ([pk_i[0] \cdot \mu + e_{i1}]_q, [a \cdot \mu + e_{i2}]_q)$;
  end for;
  return $[ct'_j, \ldots, ct'_k]$;

After decryption, we need to map the result back to the previous data space, only then the data can be recognized effectively. The mapping process of decoding is the inverse process of encoding. The internal structure of pt_n^{agg} obtained by each user i is:

$$pt_n^{agg} = A_0 \cdot x^0 + A_1 \cdot x^1 + \cdots + A_{m-1} \cdot x^{m-1}. \tag{24}$$

In which $A_i = \eta \cdot w_{i+1}$. And each user computes the aggregated model:

$$W_n^{agg} = \left[\frac{A_0}{\eta}, \frac{A_1}{\eta}, \ldots, \frac{A_{m-1}}{\eta}\right]. \tag{25}$$

V. Security Analysis and Performance Evaluation

A. Security Analysis

~~In this subsection~~, we analyze the security of EMK-BFV and SecFed. The security of SecFed is guaranteed by EMK-BFV. Our security analysis is based on the assumption that TEE is completely trustworthy. Therefore, we did not perform a security analysis for the data structures and operations in secure memory.

Definition 1: RLWE problem: Given a uniformly generated random polynomial $a \in R_q$, $s \in R_q$ and $e \in R_q$ (e obeys the distribution ξ). Distinguish $(a, a \cdot s + e)$ from (a, x), where x is randomly selected.

Theorem 1: The EMK-BFV can be shown to be semantically secure under the assumption of RLWE problem.

Proof: We demonstrate the security of EMK-BFV in three aspects: key generation, encryption and ciphertext aggregation. Secure key generation ensures that an attacker has no way to infer private key information from the public key. Secure encryption and aggregation processes guarantee that an attacker cannot infer privacy from the ciphertext.

In the key generation phase, if we set $sk = s \in R_2$ as private key and $pk = [-(a \cdot s + e), a]$ as public key, due to the RLWE, distinguishing $[-(a \cdot s + e), a]$ from $[x, a]$ is difficult. We randomly divide s into $[s_1, s_2, \ldots, s_n]$ according to the rules of polynomial arithmetic. ~~Also~~, since the operation is built on a polynomial ring, addition between elements can be regarded as xor operation. Thus the relationship between s and $[s_1, s_2, \ldots, s_n]$ can be viewed as:

$$s = s_1 \oplus s_2 \oplus \cdots \oplus s_n. \tag{26}$$

Authorized licensed use limited to: Anhui University. Downloaded on March 02,2026 at 02:25:13 UTC from IEEE Xplore. Restrictions apply.

Also according to the characteristics of dividing s, we can divide the public key:

$$a \cdot s + e = a \cdot \sum_{i=0}^{n} s_i + e = (a \cdot s_1 + e) + \cdots + (a \cdot s_n + e). \quad (27)$$

By dividing the above, we can get multiple public-private key pairs: $(s_i, [-(a \cdot s_i + e), a])$. Furthermore, we can get $s_1 \oplus s_2 \oplus \cdots \oplus s_n \oplus s = 0$ from $s = s_1 \oplus s_2 \oplus \cdots \oplus s_n$ and obtain the following relation:

$$\begin{aligned} s_1 &= s_2 \oplus s_3 \oplus s_4 \oplus \cdots \oplus s_{n-1} \oplus s_n \oplus s, \\ s_2 &= s_1 \oplus s_3 \oplus s_4 \oplus \cdots \oplus s_{n-1} \oplus s_n \oplus s, \\ &\cdots \\ s_n &= s_1 \oplus s_2 \oplus s_3 \oplus \cdots \oplus s_{n-2} \oplus s_{n-1} \oplus s. \end{aligned} \quad (28)$$

Therefore the element obtained by random sampling from R_q is equivalent to the result obtained by randomly dividing the sampled elements. For $sk = s_i$ and $pk = [-(a \cdot s_i + e), a]$, due the RLWE problem, the attacker cannot compute s_i from $[-(a \cdot s_i + e), a]$ when a is known.

In the encryption phase, we obtain ciphertext $ct_i = (ct_i[0], ct_i[1])$ where:

$$\begin{aligned} ct_i[0] &= \left[-a \cdot \mu \cdot sk_i + e_i \cdot \mu + e_{i1} + \frac{q}{t} \cdot pt_i\right]_q, \\ ct_i[1] &= [a \cdot \mu + e_{i2}]_q. \end{aligned} \quad (29)$$

For $(-a \cdot \mu \cdot sk_i)$ and $(\frac{q}{t} \cdot pt_i)$ in $ct_i[0]$, they are quantitatively similar elements in the polynomial ring and the attacker cannot distinguish between them. The only way to eliminate $(-a \cdot \mu \cdot sk_i)$ from $ct_i[0]$ to obtain $(\frac{q}{t} \cdot pt_i)$ is to use the private key and $ct_i[0]$. However, the attacker cannot obtain the private key from the public key due to the RLWE problem. Therefore, the encryption is secure.

For ciphertext $[ct_a, ct_{a+1}, \ldots, ct_b]$, the form of the ciphertext after aggregation is:

$$\sum_{i=a}^{b} ct_i^{agg}[0] = \left[\sum_{i=a}^{b}\left(-a \cdot \mu \cdot sk_i + e' + \frac{q}{t} pt_i\right)\right]_q = \left[-a \cdot \mu \cdot \sum_{i=a}^{b} sk_i + e' + \frac{q}{t} pt_i\right)\Bigg]_q. \quad (30)$$

And $\sum_{i=a}^{b} sk_i = s_a + s_{a+1} + \cdots + s_b$. We have the following relationships:

$$\begin{aligned} s &= s_1 \oplus s_2 \oplus \cdots \oplus s_n, \\ s_a + \cdots + s_b &= s_1 \oplus \cdots \oplus a_{a-1} \oplus a_{b+1} \oplus \cdots \oplus s. \end{aligned} \quad (31)$$

Thus the ciphertext encrypted by s and $s_a + \cdots + s_b$ has security equivalence. Due to the RLWE problem, an attacker cannot compute the private key from the public key and is not able to crack the aggregated ciphertext.

We can conclude that the EMK-BFV is semantically secure with the assumption of RLWE problem.

Theorem 2: Security against an honest-but-curious server: In SecFed, the hones-but-curious GP in CS cannot infer any personal information from users' data.

Proof: In SecFed, TEE does not leak user privacy, and GP in CS has access to user dataset size, user ciphertext and aggregated ciphertext. First, in step 1, GP can know the size of each user dataset $[size_1, size_2, \ldots, size_n]$. However, GP cannot infer the dataset content of user i from $size_i$ and can only have a preliminary judgment about the user's data scale. Second, in step 6, user i encrypt W_i and transmit ct_i to GP, which:

$$ct_i = (ct_i[0], ct_i[1]) = \left(\left[-a \cdot \mu \cdot sk_i + e_i \cdot \mu + \frac{q}{t} \cdot W_i\right]_q, [a \cdot \mu + e_{i2}]_q\right). \quad (32)$$

The added noise in $ct_i[0]$ and $ct_i[1]$ can guarantee the security according to RLWE problem. RLWE guarantees that $ct_i[0]$ and $ct_i[1]$ are computationally indistinguishable from uniformly random elements of R_q. GP cannot remove the interfering term in the ciphertext to obtain the plaintext without the sk_i. Third, for the aggregated ciphertext:

$$\sum_{i=a}^{b} ct_i^{agg}[0] = \left[\sum_{i=a}^{b}\left(-a \cdot \mu \cdot sk_i + e' + \frac{q}{t} pt_i\right)\right]_q = \left[-a \cdot \mu \cdot \sum_{i=a}^{b} sk_i + e' + \frac{q}{t} pt_i\right)\Bigg]_q. \quad (33)$$

It has interference terms generated by $s_1 + s_2 + \cdots + s_n$. $s_1 + s_2 + \cdots + s_n$ and s have equivalence and GP cannot infer privacy from the aggregated ciphertext.

Theorem 3: Security against the channel eavesdroppers: In SecFed, the eavesdroppers monitoring the channel cannot infer user dataset or model information from the intercepted data.

Proof: The eavesdropper has access to $ct_i = (ct_i[0], ct_i[1])$ encrypted by EMK-BFV, where $ct_i[0] = [-a \cdot \mu \cdot sk_i + e_i \cdot \mu + e_{i1} + \frac{q}{t} \cdot pt_i]_q$ and $ct_i[1] = [a \cdot \mu + e_{i2}]$. In ciphertext, the $(a \cdot \mu \cdot sk_i)$, $(\frac{q}{t} \cdot pt_i)$ and $(a \cdot \mu + e_{i2})$ are polynomials with larger coefficients, so eavesdropper cannot identify the polynomial $(\frac{q}{t} \cdot pt_i)$ from which the plaintext pt_i is contained. The attacker can only get the plaintext pt_i by removing the interfering items if he has the private key.

Eavesdroppers can also trap public keys, which $pk_i = (pk_i[0], pk_i[1]) = ([-(a \cdot sk_i + e_i)]_q, a)$. Due to the difficulty of RLWE problem, it is difficult for an eavesdropper to infer sk_i when pk_i is known. Specifically, we cannot distinguish between $-(a \cdot sk_i + e_i)$ and a due to the addition of noise. Nor can we use a to obtain sk_i.

Theorem 4: Security against honest-but-curious user: In SecFed, the hones-but-curious user cannot infer any personal information by eavesdropping the shared data from other users.

Proof: For the hones-but-curious user i, he knows s, s_i and $s = s_1 + s_2 + \cdots + s_n$. If user i wants to know s_j, then he must get $[s_1, s_2, \ldots, s_{j-1}, s_{j-2}, \ldots, s_n, s]$. However, user i has only

Authorized licensed use limited to: Anhui University. Downloaded on March 02,2026 at 02:25:13 UTC from IEEE Xplore. Restrictions apply.

s and s_i. User i can compute s_j only if user i is complicit with all users except user j. EMK-FL is able to resist collusion attacks up to N-1 users. An honest but curious user cannot infer the privacy of other users. □

Theorem 5: Security against collusion attack between users and GP in server: In SecFed, collusion between users and GP in server without TEE is incapable of causing leakage to users' sensitive dataset.

Proof: We consider TEE to be trusted in our security model and have also taken some measures to guard against side-channel attacks to prevent information leakage. Therefore collusion may occur between users and GP in server.

Users & GP in server: GP has ciphertexts $ct_1, \ldots, ct_n$ encrypted by individual users using $s_1, \ldots, s_n$ respectively. After aggregation GP gets the aggregated ciphertext ct_n^{agg}. One or some users (less than N-1) will share $s_i \cdots, s_j$ and s when they collude with GP. With the information shared by these compromised users, the GP can decrypt ct_n^{agg} as well as the ciphertexts of each compromised user $ct_i, \ldots, ct_j$, but it cannot decrypt the encrypted information of the users who have not colluded with the GP. This type of collusion allows the GP to obtain the plaintext of ct_n^{agg}. However, the GP cannot infer the encrypted information of a particular user from the aggregated plaintext pt_n^{agg}. Although the GP sells the aggregation results to the outsiders, it does not violate the privacy of the honest users.

B. ~~Performance Evaluation~~

In this section, we tested and evaluated the performance of EMK-BFV and SecFed, respectively. Furthermore, a series of comparative tests were conducted to demonstrate performance.

1) Methodology: We implement our scheme and test their performance with the following settings:

Hardware Setting: The evaluation are performed on a desktop with 11th Gen Intel(R) Core(TM) i7-11700 @ 2.50 GHz processor and 32 GB of running memory. In addition, NVIDIA ~~GeForce GTX 1650 SUPER is also used to speed up the model training process in FL.~~

~~*OS and Software Setting:*~~ We use Ubuntu Server 18.04 LTS as CS operating system and Ubuntu 18.04 LTS as user operating system. We construct EMK-BFV based on the crypto library Mircosoft SEAL [40] with version 3.7.

~~*TEE Setting:*~~ We use Intel SGX as TEE technology for testing, which provides a secure area in memory called Enclave for running functions in SecFed. The SGX driver version is 2.11 while SGX SDK version 2.12. Side-channel attacks [41], [42] have the potential to result in privacy leakage of data in SGX. But some countermeasures (such as noise injection [43]) are also proposed, which can be used in SecFed. ~~Herein,~~ we conducted an experiment to show the effectiveness of noise injection to weaken the side-channel attack in SecFed. In this experiment, we randomly reiterate specific functions without storing the results as stabilizer, effectively ensuring a sustained level of power consumption. As shown in Fig. 7, by repeating the execution of some functions, it is possible to stabilize the processor 16 cores power within a certain range. Although this method incurs additional energy consumption, it prevents attackers from analyzing information on changes in power consumption.

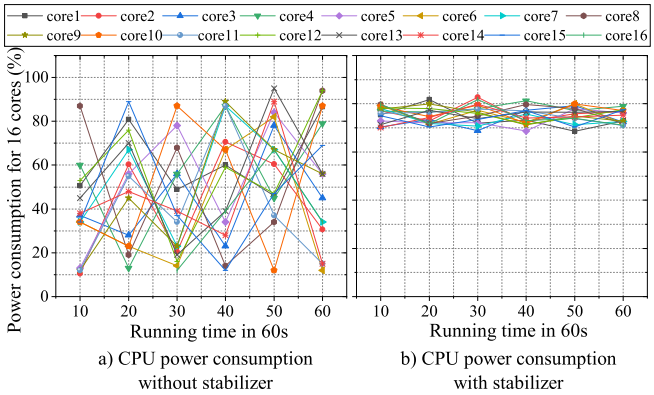


Fig. 7. Power consumption of 16 Cores in CPU.

~~TABLE IV~~
DATASETS

Dataset	Input Size	Sample Num	Parition
FEMINIST	28x28	805263	3500 users
EMINIST	28x28	814255	Custom
FashionMINIST	28x28	70000	Custom

~~*Training Setting:*~~ We use three datasets FEMINIST [44], EMINIST [45] and FashionMINIST [46] as shown in Table IV for training models. We use the Fully Connect Neural Network (CNN) as model. The input layer consists of 784 nodes. The output layer uses softmax activation functions. The hidden layer is a five-layer structure with ReLu activation function. Our model is trained with Adam optimizer [47] at a learning rate of 0.01 with 20 local epochs in one aggregation round.

~~*Parameter Setting:*~~ About encryption and decryption, we choose 1024 as the degree of irreducible polynomial, where $f(x) = x^{1024} + 1$. We select 10^5 as the plaintext space coefficient modulo and select 2^{16} as the ciphertext space coefficient modulo. We set ξ to a discrete distribution on ciphertext space.

2) Performance Evaluation of EMK-BFV: Herein, we evaluated and compared EMK-BFV with traditional BFV [10] and multi-key BFV scheme (mkBFV) in [48]. As shown in Fig. 4, both BFV and EMK-BFV consist of five modules: Key Generation, Encryption & Decryption, Aggregation, Noise Refresh, and Multiplication. ~~Hence~~, we tested their performance in each module to show its high effiency as follows:

~~*Key Generation:*~~ ~~First,~~ we tested the time cost of key generation in three schemes with various number of users. Fig. 8(a) shows the overall time, while Fig. 8(b) shows the time cost of each function in key generation. As shown in Fig. 8(b), the time cost of our new function $secretGen()$ for supporting multi-user scenarios does not increase with the number of users, which is efficient enough. The function $secureMul()$ in EMK-BFV circumvents the construction of relinearization key rlk, which greatly shortens the key construction time. Though mkBFV also applies relinearization to speed up key generation, our

Authorized licensed use limited to: Anhui University. Downloaded on March 02,2026 at 02:25:13 UTC from IEEE Xplore. Restrictions apply.

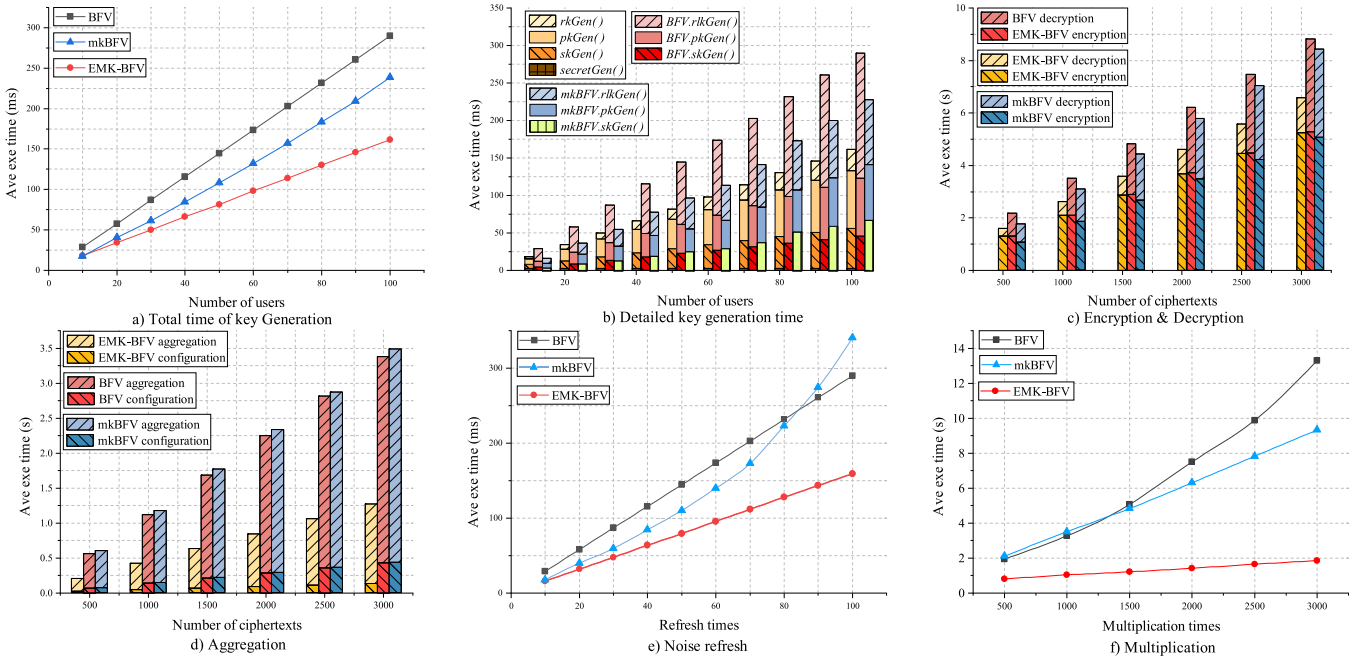


Fig. 8. Performance comparison of EMK-BFV and existing work.

EMK-BFV is still more efficient than both BFV and mkBFV. With the increase of users, the advantage of EMK-BFV becomes more obvious in Key Generation.

~~Encryption & Decryption~~: Second, ~~we tested and compared their time cost in encryption & decryption with different number of outsourced ciphertexts for aggregation~~. From Fig. 8(c), we can find that the time cost increases linearly with the number of ciphertexts. Though both BFV and EMK-BFV have the same design and programming implementation, EMK-BFV is still slightly better than BFV because the ciphertext in BFV has a simpler structure. mkBFV has a slight advantage when it comes to encryption. But EMK-BFV simplifies the ciphertext structure which leads to higher efficiency in decryption. This makes the overall efficiency of EMK-BFV in encryption and decryption better than that of mkBFV. In addition, we tested the communication cost of EMK-BFV with different numbers of ciphertexts. When the number of ciphertexts achieves 3000, it takes about 6.5 s and 350 KB, which is efficient and acceptable for FL.

~~Aggregation:~~ Third, we tested the aggregation time in GP with different number of ciphertexts. As shown in Fig. 8(d), with the increase of ciphertext numbers, the time cost of EMK-BFV slowly increases, which is much more efficient than both BFV and mkBFV.

~~Noise Refresh~~: Finally, we conducted three experiments to show the efficiency and superiority of noise refresh in SedFed. We first tested the time cost to implement different number of noise refresh in three schemes. BFV controls the noise growth by modulus switching, and mkBFV achieves noise refresh by Bootstrapping technique. Noise refresh in EMK-BFV is accomplished by executing $noiseRef()$. Fig. 8(e) shows that noise refresh in EMK-BFV is still more efficient than that in BFV and mkBFV. The implementation of $noiseRef()$ needs to exchange data between GP and TEE, and the communication overhead of this part has been tested and included in the execution time of $noiseRef()$.

TABLE V
STATISTICS ON $noiseRef()$ EXECUTION

NAC	Maximum	NE	Percentage	TNI
1000	2 ciphertexts	94	29%	328
	3 ciphertexts	124	38%	
	4 ciphertexts	110	33%	
2000	2 ciphertexts	173	26%	659
	3 ciphertexts	290	44%	
	4 ciphertexts	196	30%	
3000	2 ciphertexts	287	29%	1001
	3 ciphertexts	430	43%	
	4 ciphertexts	284	28%	

NAC: Number of Aggregated Ciphertexts; Maximum: Maximum number of aggregations allowed at one time; NE: Number of Executions; TNI: Total Number of Invocations.

We statistically recorded the number of noise refresh to be performed while aggregating different number of ciphertexts: 1000, 2000, and 3000. Experiments show that the noise budget is usually exhausted within two, three or four aggregations. As shown in Table V, the total number of invocations on noise refresh is linearly to the number of ciphertexts and more than 1000 times of noise refresh should be executed with 3000. Hence, our design of $noiseRef()$ significantly enhances the efficiency of SecFed.

Authorized licensed use limited to: Anhui University. Downloaded on March 02,2026 at 02:25:13 UTC from IEEE Xplore. Restrictions apply.

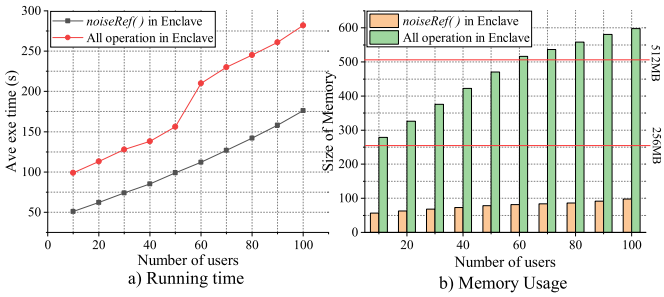


Fig. 9. Enclave runtime and memory usage.

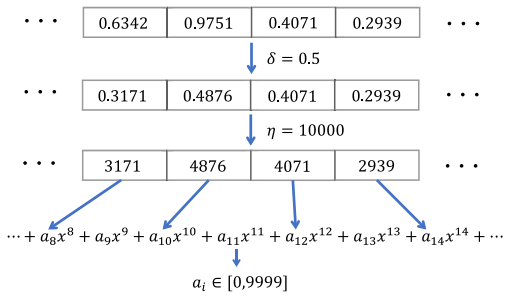


Fig. 10. Encoding method.

~~In addition,~~ we evaluated the execution time and memory usage of $noiseRef()$ to show its design rationality. Due to the memory constraints of Enclave size, we only put $noiseRef()$ into the Enclave instead of all operations on the cloud side. In this experiment, we showed the time and memory usage gap between only noise refreshing in Enclave and putting all operations into Enclave under different number of users with model size 10^5. As shown in the Fig. 9, putting all operations into the Enclave will exceed the size of Enclave (256 MB) and result in the expensive page exchange mechanism. Especially when the memory usage exceeds the size of two Enclaves (number of users=60, memory usage>512 MB), the runtime spikes. However, the memory cost of $noiseRef()$ grows slowly and remains considerably lower than the size of Enclave.

Multiplication: ~~Finally,~~ we tested the multiplication time of three schemes. ~~In our design~~, EMK-BFV avoids the time-consuming operation of relinearization in BFV. From Fig. 8(f), we can see that the time for EMK-BFV multiplication is linear to the number of multiplications. With the increase of multiplication times, EMK-BFV performs better than both BFV and mkBFV.

3) Performance Evaluation of SecFed: In this section, we first compared SecFed with a single-key HE-based FL scheme [8] to evaluate the additional time overhead to achieve multi-user privacy protection. Then we evaluated the performance of SecFed in terms of computation overhead, communication overhead and offline protection mechanism. ~~Furthermore,~~ we compared it with TEE-based FL [49] and xMK-CKKS [34] in terms of memory consumption, overall time and model accuracy. We use the encoding method as in Fig. 10 and zero-bit padding to ensure correct encryption and prevent overflow, which allows to fully preserve the floating-point information.

TABLE VI
SECFED VS PFMLP

Scheme	Key Size	Model Size	Client Num	Five Rounds Total	Ten Rounds Total
SecFed	1024	10^5	10	312s	534s
PFMLP				298s	507s
SecFed	1024	10^5	20	368s	641s
PFMLP				351s	609s
SecFed	2048	10^5	10	486s	853s
PFMLP				462s	817s
SecFed	2048	10^5	20	538s	948s
PFMLP				513s	910s

1) Comparison with Single-key HE: Comparison with Single-key HE: Compared to single-key HE-based FL frameworks, SecFed incurs some additional computational overhead for protecting multi-user privacy. To clearly demonstrate the additional overhead introduced by SecFed, we compared it with an advanced and efficient single-key HE privacy-preserving FL framework (PFMLP [8]) on MNIST dataset. As illustrated in Table VI, we conducted tests on the runtime of SecFed and PFMLP under varying conditions, including different key lengths, client counts, and numbers of rounds. It can be observed that SecFed achieves multi-user privacy protection with an extra 5% time overhead. This further substantiates that the utilization of TEE can enhance the operational efficiency of SecFed, bringing it close to the efficiency of FL based on single-key HE.

2) Computation Overhead of SecFed: We tested the time cost in users and CS. Since the time consumption of data transmission between users and CS is affected by network environment, we did not demonstrate it in our experiments. As shown in Fig. 11(a), we tested the time for users and CS to run SecFed with different number of CNN weights (model size) from 10^3 to 10^7. Users need to perform $Encoding()$, $Enc()$, $Dec()$ and $Decoding()$, which brings few time overhead compared to model training as shown in Fig. 11(a) The running time of SecFed in CS is related to model size. When the size is larger, the number of ciphertexts to be processed by CS is higher. As shown in Fig. 11(b), we likewise tested the time taken for CS to perform SecFed with different model sizes. The time for CS to perform SecFed increases with the size of model.

3) Communication Overhead of SecFed: We further evaluated the total communication overhead of one round in SecFed. The data transferred from users to CS is mainly the encrypted model. We tested the communication cost when the scale of model parameters ranges from 10^4 to 10^6. Table VII shows the number of ciphertext and communication consumption corresponding to the number of model parameters. When the model size reaches 7027860, the communication cost is about 30.2 MB, which is acceptable.

4) Offline Protection Mechanism Efficiency of SecFed: In this test, the default total number of users is 3000. We tested the

Authorized licensed use limited to: Anhui University. Downloaded on March 02,2026 at 02:25:13 UTC from IEEE Xplore. Restrictions apply.

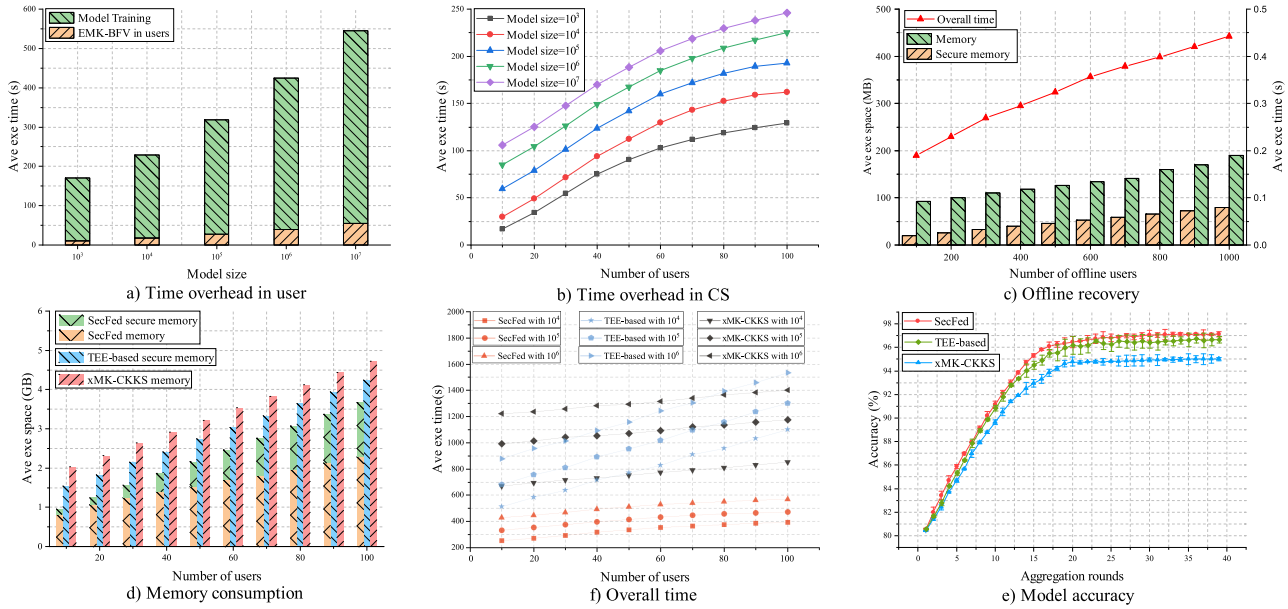


Fig. 11. Performance of SecFed.

TABLE VII
DATASETS

Model Size	Ciphertexts Num	Communication Overhead
50670	61	220KB
616420	750	2.6MB
7027860	8650	30.2MB

TABLE VIII
INFORMATION ABOUT ENCLAVE

Number of Users	Enclave Space Allocation		Total Size	Creation Time
	Computing Tasks	SGX Configuration		
200	130.34MB	15.58MB	145.92MB	8.67s
400	152.47MB	17.97MB	170.44MB	11.07s
600	173.81MB	21.35MB	195.16MB	13.93s
800	196.07MB	23.84MB	219.91MB	16.74s
1000	218.93MB	26.72MB	245.65MB	19.23s

Each ciphertext $\leq 61bytes$; Each key $\leq 19bytes$;
Each function in Enlcave $\leq (50KB + N \times 100bytes)$.

additional overhead incurred by offline protection mechanism with a different number of offline users from 100 to 1000. From Fig. 11(c), it can be observed that the time cost is linear with the number of offline users. But it only takes about 0.25 s even when 1000 users are offline, which can be ignored compared with the time of SecFed in CS. Also the usage of secure memory is within a reasonable range and it grows linearly with the number of offline users.

5) Comparison of Memory Consumption: Compared to SecFed, the TEE-based FL and xMK-CKKS are extremes in both directions. The TEE-based FL migrates the entire privacy process into secure memory to complete. xMK-CKKS, on the other hand, utilizes multi-key HE running in regular memory to protect privacy. SecFed migrates some of the operations to secure memory, which effectively reduces memory overhead. As shown in Fig. 11(d), we tested the memory overhead of all three. The sum of secure memory and normal memory used by SecFed is smaller than the secure memory of the TEE-based FL and the normal memory of xMK-CKKS. In particular, SecFed avoids the expensive page swapping mechanism in the use of secure memory.

We also evaluated the memory usage and time cost of SecFed in Enclave with different number of users. With model size 10^5, we tested the maximum usage and creation time per Enclave in one round for a number of users from 200 to 1000. According to Table VIII, even with 1000 users, the memory overhead remains below the limitation of Enclave (256 MB). Generally, it is sufficient to meet the requirements of practical application scenarios.

6) Comparison of Overall Time: In SecFed, we put some operations into TEE to reduce the time overhead. To demonstrate the speedup effect more clearly, we tested the time of the TEE-based FL and xMK-CKKS with different model size. As shown in Fig. 11(f), the TEE-based FL moves all operations into TEE, which will result in excessive time consumption compared to SecFed. This time consumption will increase further with the number of users, which is mainly due to the large number of data migration and encryption/decryption operations in the TEE. The xMK-CKKS has some advantages over the TEE-based FL for a small number of users and small model size, but the time overhead of both is larger compared to SecFed. SecFed performs excellently in encryption and aggregation scenarios of

Authorized licensed use limited to: Anhui University. Downloaded on March 02,2026 at 02:25:13 UTC from IEEE Xplore. Restrictions apply.

large-scale data, which indicates the suitability of SecFed to be employed for FL in large data-holder scenarios, cross-silo FL. However, in large-amount devices with small-amount data FL scenarios (cross-silo FL [50], [51]), the frequent execution of encryption processes could affect the performance of SecFed.

7) Comparison of Model Accuracy: Finally, we evaluated the model accuracy of SecFed and compared it with TEE-based FL and xMK-CKKS. In SecFed, using EMK-BFV to encrypt data does not introduce errors. However, the encryption method in xMK-CKKS and the migration in TEE-based FL may incur some errors. Because CKKS [30] takes an approximate calculation in encryption and some fractional parts are discarded during migration in TEE-based FL. As shown in Fig. 11(e), with the increase of training rounds, the accuracy becomes stable, and our SecFed achieves a higher accuracy. Also we can find that the error introduced by the TEE-based FL is smaller than that of xMK-CKKS, but both are larger than that of SecFed.

VI. Conclusion

In this paper, we utilized TEE to design SecFed, an efficient and secure federated learning framework in multi-user scenarios. We designed a secure multi-key homomorphic encryption cryptosystem based on TEE to preserve user privacy and data security. In order to solve the problem of user offline, we proposed an offline protection mechanism to ensure collaborated training normally. Security analysis and performance evaluation demonstrate SecFed's security and efficiency. In practical applications, the features of SecFed match cross-silo FL, which contains large-scale data-holders. SecFed has a better performance in cross-silo FL compared to cross-device FL which has data-holders with small amount of data and need multiple encryptions. In the future, we will further improve the efficiency of SecFed and explore significant applications.

References

[1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84–90, 2017.

[2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv: 1810.04805*.

[3] H.-T. Cheng et al., "Wide & deep learning for recommender systems," in *Proc. 1st Workshop Deep Learn. Recommender Syst.*, 2016, pp. 7–10.

[4] B. Zhao, K. R. Mopuri, and H. Bilen, "iDLG: Improved deep leakage from gradients," 2020, *arXiv: 2001.02610*.

[5] A. Acar, H. Aksu, A. S. Uluagac, and M. Conti, "A survey on homomorphic encryption schemes: Theory and implementation," *ACM Comput. Surv.*, vol. 51, no. 4, pp. 1–35, 2018.

[6] M. Naehrig, K. Lauter, and V. Vaikuntanathan, "Can homomorphic encryption be practical?," in *Proc. 3rd ACM Workshop Cloud Comput. Secur. Workshop*, 2011, pp. 113–124.

[7] A. López-Alt, E. Tromer, and V. Vaikuntanathan, "On-the-fly multiparty computation on the cloud via multikey fully homomorphic encryption," in *Proc. 44th Annu. ACM Symp. Theory Comput.*, 2012, pp. 1219–1234.

[8] H. Fang and Q. Qian, "Privacy preserving machine learning with homomorphic encryption and federated learning," *Future Internet*, vol. 13, no. 4, 2021, Art. no. 94.

[9] J. Park and H. Lim, "Privacy-preserving federated learning using homomorphic encryption," *Appl. Sci.*, vol. 12, no. 2, 2022, Art. no. 734.

[10] J. Fan and F. Vercauteren, "Somewhat practical fully homomorphic encryption," *Cryptol. ePrint Arch.*, Paper 2012/144, 2012. [Online]. Available: https://eprint.iacr.org/2012/144

[11] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *Proc. IEEE Symp. Secur. Privacy*, 2017, pp. 3–18.

[12] R. Canetti, U. Feige, O. Goldreich, and M. Naor, "Adaptively secure multiparty computation," in *Proc. 28th Annu. ACM Symp. Theory Comput.*, 1996, pp. 639–648.

[13] D. Pasquini, D. Francati, and G. Ateniese, "Eluding secure aggregation in federated learning via model inconsistency," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2022, pp. 2429–2443.

[14] K. Bonawitz et al., "Practical secure aggregation for privacy-preserving machine learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 1175–1191.

[15] J. H. Bell, K. A. Bonawitz, A. Gascón, T. Lepoint, and M. Raykova, "Secure single-server aggregation with (poly) logarithmic overhead," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2020, pp. 1253–1269.

[16] M. Hao, H. Li, G. Xu, S. Liu, and H. Yang, "Towards efficient and privacy-preserving federated deep learning," in *Proc. IEEE Int. Conf. Commun.*, 2019, pp. 1–6.

[17] Y. Liu, T. Fan, T. Chen, Q. Xu, and Q. Yang, "FATE: An industrial grade platform for collaborative learning with data protection," *J. Mach. Learn. Res.*, vol. 22, no. 226, pp. 1–6, 2021.

[18] C. Dwork, "Differential privacy: A survey of results," in *Proc. Int. Conf. Theory Appl. Models Computation*, 2008, pp. 1–19.

[19] S. Augenstein et al., "Generative models for effective ML on private, decentralized datasets," 2019, *arXiv: 1911.06679*.

[20] K. Wei et al., "Federated learning with differential privacy: Algorithms and performance analysis," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 3454–3469, 2020.

[21] M. Abadi et al., "Deep learning with differential privacy," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2016, pp. 308–318.

[22] R. Hu, Y. Guo, H. Li, Q. Pei, and Y. Gong, "Personalized federated learning with differential privacy," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 9530–9539, Oct. 2020.

[23] H. Xiao, Q. Zhang, Q. Pei, and W. Shi, "Privacy-preserving neural network inference framework via homomorphic encryption and SGX," in *Proc. IEEE 41st Int. Conf. Distrib. Comput. Syst.*, 2021, pp. 751–761.

[24] A. Mondal, Y. More, R. H. Rooparaghunath, and D. Gupta, "FLATEE: Federated learning across trusted execution environments," 2021, *arXiv:2111.06867*.

[25] F. Mo, H. Haddadi, K. Katevas, E. Marin, D. Perino, and N. Kourtellis, "PPFL: Privacy-preserving federated learning with trusted execution environments," in *Proc. 19th Annu. Int. Conf. Mobile Syst. Appl. Serv.*, 2021, pp. 94–108.

[26] Z. Yang, S. Hu, and K. Chen, "FPGA-based hardware accelerator of homomorphic encryption for efficient federated learning," 2020, *arXiv: 2007.10560*.

[27] C. Zhang, S. Li, J. Xia, W. Wang, F. Yan, and Y. Liu, "{BatchCrypt }: Efficient homomorphic encryption for { Cross-Silo} federated learning," in *Proc. USENIX Annu. Tech. Conf.*, 2020, pp. 493–506.

[28] A. Madi, O. Stan, A. Mayoue, A. Grivet-Sébert, C. Gouy-Pailler, and R. Sirdey, "A secure federated learning framework using homomorphic encryption and verifiable computing," in *Proc. Reconciling Data Analytics Automat. Privacy Secur.: A Big Data Challenge*, 2021, pp. 1–8.

[29] D. Stripelis et al., "Secure neuroimaging analysis using federated learning with homomorphic encryption," in *17th Int. Symp. Med. Inf. Process. Anal.*, 2021, pp. 351–359.

[30] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *Proc. Int. Conf. Theory Application Cryptol. Inf. Secur.*, 2017, pp. 409–437.

[31] W. Ou, J. Zeng, Z. Guo, W. Yan, D. Liu, and S. Fuentes, "A homomorphic-encryption-based vertical federated learning scheme for rick management," *Comput. Sci. Inf. Syst.*, vol. 17, no. 3, pp. 819–834, 2020.

[32] Y. Liu, Y. Kang, C. Xing, T. Chen, and Q. Yang, "A secure federated transfer learning framework," *IEEE Intell. Syst.*, vol. 35, no. 4, pp. 70–82, Jul./Aug. 2020.

[33] E. Hosseini and A. Khisti, "Secure aggregation in federated learning via multiparty homomorphic encryption," in *Proc. IEEE Globecom Workshops*, 2021, pp. 1–6.

[34] J. Ma, S.-A. Naas, S. Sigg, and X. Lyu, "Privacy-preserving federated learning based on multi-key homomorphic encryption," *Int. J. Intell. Syst.*, vol. 37, pp. 5880–5901, 2022.

Authorized licensed use limited to: Anhui University. Downloaded on March 02,2026 at 02:25:13 UTC from IEEE Xplore. Restrictions apply.

[35] H. Tian, F. Zhang, Y. Shao, and B. Li, "Secure linear aggregation using decentralized threshold additive homomorphic encryption for federated learning," 2021, *arXiv:2111.10753*.
[36] C. Gentry, "Fully homomorphic encryption using ideal lattices," in *Proc. 41st Annu. ACM Symp. Theory Comput.*, 2009, pp. 169–178.
[37] S. Fei, Z. Yan, W. Ding, and H. Xie, "Security vulnerabilities of SGX and countermeasures: A survey," *ACM Comput. Surv.*, vol. 54, no. 6, Jul. 2021, Art. no. 126, doi: 10.1145/3456631.
[38] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," 2017, *arXiv: 1706.06083*.
[39] C. Lyu, K. Huang, and H.-N. Liang, "A unified gradient regularization family for adversarial examples," in *Proc. IEEE Int. Conf. Data Mining*, 2015, pp. 301–309.
[40] "SEAL Microsoft (release 3.7)," Mar. 2022. [Online]. Available: https://github.com/Microsoft/SEAL
[41] R. Spreitzer, V. Moonsamy, T. Korak, and S. Mangard, "Systematic classification of side-channel attacks: A case study for mobile devices," *IEEE Commun. Surv. Tut.*, vol. 20, no. 1, pp. 465–488, First Quarter 2017.
[42] G. Joy Persial, M. Prabhu, and R. Shanmugalakshmi, "Side channel attack-survey," *Int. J. Adv. Sci. Res. Rev*, vol. 1, no. 4, pp. 54–57, 2011.
[43] A. Nilsson, P. N. Bideh, and J. Brorsson, "A survey of published attacks on intel SGX," 2020, *arXiv: 2006.13598*.
[44] S. Caldas et al., "LEAF: A benchmark for federated settings," 2018, *arXiv: 1812.01097*.
[45] G. Cohen, S. Afshar, J. Tapson, and A. van Schaik, "EMNIST: An extension of MNIST to handwritten letters," 2017, *arXiv: 1702.05373*.
[46] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-mnist: A novel image dataset for benchmarking machine learning algorithms," 2017, *arXiv:1708.07747*. [Online]. Available: https://api.semanticscholar.org/CorpusID:702279
[47] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.
[48] H. Chen, W. Dai, M. Kim, and Y. Song, "Efficient multi-key homomorphic encryption with packed ciphertexts with application to oblivious neural network inference," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2019, pp. 395–412.
[49] A. A. Messaoud, S. B. Mokhtar, V. Nitu, and V. Schiavoni, "Gradsec: A tee-based scheme against federated learning inference attacks," in *Proc. 1st Workshop Syst. Challenges Reliable Secure Federated Learn.*, 2021, pp. 10–12.
[50] B. Liu, N. Lv, Y. Guo, and Y. Li, "Recent advances on federated learning: A systematic survey," 2023, *arXiv:2301.01299*.
[51] M. Aledhari, R. Razzak, R. M. Parizi, and F. Saeed, "Federated learning: A survey on enabling technologies, protocols, and applications," *IEEE Access*, vol. 8, pp. 140699–140725, 2020.



Yuxuan Cai received the BEng degree in computer science and technology from Chongqing Technology and Business University, Chongqing, China, in Jun. 2021. He is currently working toward the master's degree with the School of Cyber Engineering, Xidian University. His research interests include privacy-preserving federal learning, homomorphic encryption, and SGX.



Wenxiu Ding (Member, IEEE) received the BEng and PhD degrees in information security from Xidian University, Xi'an, China, in 2012 and 2017. She was the research assistant with the School of Information Systems, Singapore Management University from 2015 to 2016. Now, she is an associate professor with the School of Cyber Engineering, Xidian University. Her research interests include privacy preservation, access control, and trust management.



Yuxuan Xiao received the BEng degree in information security from Xidian University, Xi'an, China, in Jun. 2022. He is currently studying with the School of Cyber Engineering, Xidian University. His main research interests include privacy protection and communication efficiency for Federal learning.



Zheng Yan (Senior Member, IEEE) received the BEng degree in electrical engineering and the MEng degree in computer science and engineering from the Xi'an Jiaotong University, Xi'an, China, in 1994 and 1997, respectively, the second MEng degree in information security from the National University of Singapore, Singapore, in 2000, and the licentiate of science and the doctor of science in technology in electrical engineering from the Helsinki University of Technology, Helsinki, Finland. She is currently a professor with the Xidian University, Xi'an, China. Her research interests include trust, security, privacy, and security-related data analytics. She serves as a general or program chair for more than 30 international conferences and workshops. She is a steering committee co-chair of IEEE Blockchain international conference. She is also an associate editor of many reputable journals, e.g., *IEEE Internet of Things Journal*, *Information Sciences*, *Information Fusion*, JNCA, IEEE Access, SCN, etc.



Ximeng Liu (Senior Member, IEEE) received the BSc degree in electronic engineering and the PhD degree in cryptography from Xidian University, Xi'an, China, in 2010 and 2015, respectively. He is currently a full professor with the College of Computer Science and Data Science, Fuzhou University. Also, he was a research fellow with the Peng Cheng Laboratory, Shenzhen, China. He has published more than 200 papers on the topics of cloud security and Big Data security, including papers in *IEEE Transactions on Computers*, *IEEE Transactions on Industrial Informatics*, *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Services Computing*, and *IEEE Internet of Things Journal*. He was awarded the "Minjiang Scholars" distinguished professor, "Qishan Scholars" in Fuzhou University, and ACM SIGSAC China Rising Star Award in 2018. His research interests include cloud security, applied cryptography, and Big Data security.



Zhiguo Wan (Member, IEEE) received the BS degree in computer science from Tsinghua University, in 2002, and the PhD degree from the School of Computing, National University of Singapore, Singapore, in 2007. He was an associate professor with the School of Computer Science and Technology, Shandong University. From 2008 to 2014, he was an assistant professor with the School of Software, Tsinghua University. He was a postdoctoral researcher with the Katholieke University of Leuven, Belgium, from 2006 to 2008. He is currently a principal investigator with Zhejiang Laboratory, Hangzhou, Zhejiang, China. His research interests include security and privacy for blockchain, cloud computing, and intelligent computing.

Authorized licensed use limited to: Anhui University. Downloaded on March 02,2026 at 02:25:13 UTC from IEEE Xplore. Restrictions apply.